NYU TANDON

MODULE 2 · LAB 2

AUTOMATE THE REPEATABLE PART

Build it, break it, present it.

IE-GY 9113B · Systems Integration: From ERP to Agentic AI

Friday, October 9, 2026 · 8:00–10:30 AM · 2 MTC Room 802

The shape of the session

A short open and a visit from the field, then you work straight through — we don't stop and regroup — and finish with presentations.

8:00

OPEN + GATE

~10 min

Gate check; recall your Lab 1 candidate, sandbox, and seeded flaws.

→

8:10

GUEST

15 min

A view from industry — a real problem from the field, plus your questions.

→

8:25

THE WORK AHEAD

~10 min

Walk through the four tasks and the time budget before you start.

→

8:35

WORK BLOCK

~60 min

Refine → break → harden → ROI. Continuous and self-paced — no reconvene.

→

9:35

PRESENT & SHARE

~45 min

7-min show + 3-min feedback, with an industry guest on the board.

THE GATE. You come in with a first-pass automation on your sandbox, and today you refine it, break it on your own seeded flaws, and present it. Couldn't get it working? Bring what you have and the exact error you hit — the start of the work block is a recovery path to a minimal version (backup).

8:10

A view from industry



Christie Subirats

Business Improvement and Strategic Change Leader · BP

Our guest shares a real problem from her work — then takes your questions.

~10 MINUTES, THEN 5 FOR YOUR QUESTIONS. Listen for how the same ideas you're working today show up in a live operation.

Your four tasks — and the clock

We go over all four now, then you work them straight through on your own clock. We don't stop and regroup — the budget below keeps you on pace to present at 9:35.

①	**REFINE**	**~15 min**	Get the happy path clean on your sandbox — the right rows trigger, the rule splits them, the output lands.
②	**BREAK IT**	**~20 min**	Feed it your Lab 1 seeded flaws and find where it fails. This is the heart of the lab.
③	**HARDEN**	**~10 min**	For each break, name the human oversight that would catch it.
④	**ROI + PREP**	**~15 min**	Size the cost removed, then get ready to present.

Suggested budget · ~60 minutes total · the task slides that follow stay up for reference while you work.

The automation, built and broken

MIRROR, NOT TEMPLATE. Your company and candidate differ — the point is the shape. All figures synthetic.

The automation (trigger → condition → action)

Trigger · System 1. Watch the dock-status sandbox rows.

Condition · the rule. available_bikes < 20% of capacity → flag. This is your Lab 1 stable-rules decision point, now in software.

Action · System 2. Write the station ID + count to a Tasks sheet, or send an alert.

The break (feed the Lab 1 seeded flaw)

Feed the stale nightly-status rows. The bot flags on yesterday’s numbers and fires when it shouldn’t.

What it reveals. A surface bot inherits bad data — it papers over a missing integration, it doesn’t fix one.

The oversight it implies. A freshness check — confirm the live feed, not the nightly one, before acting.

Synthetic worked example — a mirror, not a template.

~15 min | 8:35

Get the happy path clean

Run your scenario on your sandbox and confirm the routine cases flow. Stuck? This is where the recovery path gets you to a working first pass (backup).

✓ The right rows trigger

The watch / search module returns the rows the automation should act on — you can see the real field values.

✓ The rule splits them

The Router / Filter — your Lab 1 decision point, now in software — sends the right rows down the right branch.

✓ The output lands

A real row, email, or message appears in System 2, populated with real values from your sandbox.

SANITY-CHECK THE OUTPUT. A handful of rows should flag, not most. If nearly everything flags, the threshold is too loose. If a 90%-full station reads “empty,” the rule is backwards. A bot applies a wrong rule with perfect confidence — validating against sense is part of the build.

~20 min | 8:50

Feed it the flaws you seeded

Lab 1 planted the bad data precisely so Lab 2 could trip on it. Your job is not to make it work perfectly — it is to find where it breaks.

Feed it this	What should happen	What it reveals
Rejection / rework rows (Status = Rejected / Rework)	Mis-routed, silently skipped, or the output fires anyway on stale data.	The bot follows its rule with perfect confidence even when the case is wrong for it — no judgment, no doubt.
Legacy-mismatch rows (the non-reconciling source)	It acts on the wrong number — triggers when it shouldn't, or misses a case it should catch.	A surface bot inherits bad data; it papers over a missing integration, it doesn't fix one.
A malformed / missing field (a blank where the rule expects a value)	An error, or a silent pass with an empty output.	Where input validation and a human checkpoint belong.

DOCUMENT EACH ONE: what you fed it, what it did, and what a person would have caught. That documentation is the deliverable — not a flawless demo.

~10 min

9:10

Name the oversight each break implies

For each break, state the specific human checkpoint that would have caught it — the human-in-the-loop discipline from Week 5, made concrete. You name it precisely; you need not build it.

A confidence / validation gate

If a required field is blank or out of range, stop and flag rather than send.

An exception route

Route rejected / rework rows to a human queue, not the auto-approve branch.

A freshness / reconciliation check

Confirm the source is the live feed, not the nightly legacy one, before acting.

“RULES-BASED” NEVER MEANS “NO HUMAN NEEDED.” A bot follows a wrong rule confidently, so route edge cases, high-value transactions, and irreversible actions to a human on purpose.

~15 min | 9:20

A method, not a number we give you

Your board memo lives on an honest estimate — so build one from your own scenario, then get ready to present.

step volume × **time per instance** × **labour rate** = recurring cost removed

Put against it

The one-time build effort plus any licence. Leave the exception volume in — the cases that still go to a human. Express it as a rough payback period.

The checkpoint question

What would this cost if it ran undetected on 500 transactions a day? Size the break, not just the saving.

ORDER-OF-MAGNITUDE AND DEFENSIBLE. A board funds a thread outcome — cash recovered, customer wait removed — tagged to a number. An honest estimate with exceptions costed beats an optimistic one that wishes them away.

7 to show, 3 to hear — from your board deck

You present from the real capstone template, with an industry guest on the board for feedback. This is the first time you tell the full story to an audience — take it properly and balance your time across the four parts. It's the deck you'll defend in December.

Min	Deck slide	Show this
~2 min	**S3–S5 — story so far**	Company & strategic problem, and your Lab 1 diagnosis — the setup for what you built.
~1.5 min	**S6 — the automation**	What your Make scenario connects and does on the happy path — a quick demo or screenshot.
~2 min	**S7 — where it broke**	The input you fed it, the failure it produced, and the oversight it implies — the heart of it.
~1.5 min	**S7 — ROI & thread**	Your ROI method and the rough prize, tagged to a thread (Cash / Customer / Operations).

7-MIN SHOW + 3-MIN FEEDBACK PER TEAM. Tell it end to end and keep your four parts in balance — it's your first full run-through of the story you'll defend in December.

Where the rule ran out

Name the patterns in the room — the strongest breaks, and the oversight each one demands.

The break is a governance finding, not a bug

Every edge case your bot shipped confidently is a place a human belongs. A bot follows a wrong rule without doubt — that is the limit of a rules-based system, felt first-hand.

The edge case is Lab 3's judgment step

The case a human would have caught intuitively is exactly the judgment decision point you marked in Lab 1 — the step an agent takes in Lab 3. You don’t build agents here; you feel why they’re needed.

ETHICS THREAD. Your automation broke on an edge case a human would have caught intuitively. What does that say about the limits of rules — and what we risk when we remove the human at scale? Hold it; it returns in Lab 3.

Quiz 1 opens tonight — the Module 1–2 midterm

Your first graded quiz covers Modules 1–2. It's taken on Brightspace outside class, then a short 1:1 with me. A heads-up now; full details are in the syllabus.

Opens tonight · Fri, Oct 9 · 11:59 PM → **Closes Tue, Oct 13 · 11:59 PM**

Covers Modules 1–2

Everything from Weeks 1–6 — the integration backbone through automation and the Center of Excellence.

Applied, not recall

You reason through the concepts on your own project and the cases, so open materials are fine — the questions can't be answered by copying a definition. Budget ~45–60 minutes for the written part.

The 1:1 is part of the grade

Book a 15-minute 1:1 with me after you submit — submitting early is fine — and within 10 days of submitting. You walk me through one or two answers; that conversation is where depth shows.

10:25

Today completes Milestone 1

Your working automation and one-page post-mortem are the evidence for Board Memo §4 (Automation Journey), filling S6–S7 of the board deck. Lab 2 completes Milestone 1 — the diagnosis of the problem and where automation adds value.

POST TO YOUR TEAM GOOGLE WORKSPACE (end of day). A screenshot of the working workflow + a one-page post-mortem: what you built, where it broke, the oversight you would add, and your simple ROI.

NEXT WEEK — MODULE 3 OPENS. Week 7: From RPA to Intelligent Automation — the one step a rule can't take, where judgment begins. That judgment step is your Lab 3 agent.

BACKUP · REFERENCE

The scenario, module by module

One worked path — build to the outcome, not the clicks. Full detail: study note §3.

Module 1 · Trigger

Read System 1

Google Sheets “Watch / Search Rows” or Airtable “Watch Records” on your process tab.

Success: Run once returns real rows.

Module 2 · Condition

The rule = your decision point

A Router or Filter encoding your exact Lab 1 rule — every case decided, no “it depends.”

Success: rows split the right way.

Module 3 · Action

Write System 2

Gmail, Slack, or a second sheet. Map which columns fill which parts of the message or row.

Success: a real output appears, populated.

CONNECTION FIRST. Authorize the Make↔Google / Make↔Airtable connection before building any module — half of all first-build snags live here. Free tier: use Run once (don’t wait on the 15-min schedule). Stuck on a module? “Build with Maia,” then check and tighten its rule.

Known snags, and the no-build path

COMMON FIRST-BUILD SNAGS

Symptom	Likely fix
Trigger returns no rows	Connection points at the wrong sheet/table, or the filter is too narrow — test with Search Rows + a simple filter.
Connection / auth failed	Re-authorize with the account that owns the sandbox (Make→Google and Make→Airtable are separate).
Output fires, fields empty	Map each field to a value from the trigger module explicitly.
Ran out of operations	Narrow the trigger to a handful of rows; a 3–4 module run costs 3–4 credits.

The no-build fallback (~25 min)

Build the smallest possible version:

(1) Search Rows on your sandbox, filtered to ~5 rows.
(2) A Filter with one simple rule.
(3) One Add-a-Row / Send-Email action. Run once.

Then break it on one seeded flaw and name one oversight.

A MINIMAL AUTOMATION, BROKEN HONESTLY, MEETS THE DELIVERABLE. Capturing where it breaks is the lesson — known failure modes are content, not embarrassment. Don't spend the class polishing a bigger one you can't finish.